

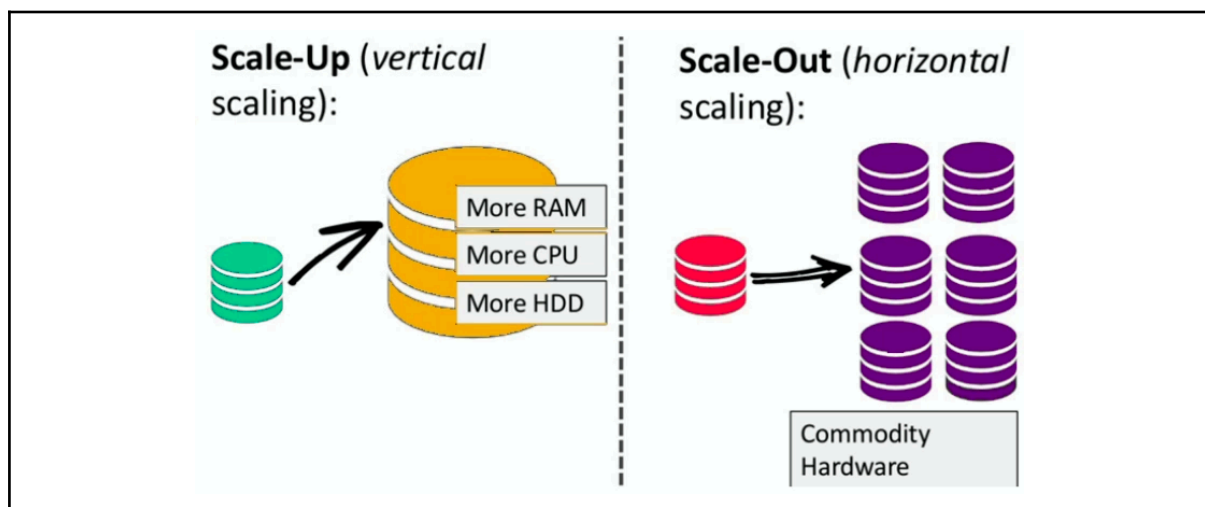
Section 7: Storage in System Design

Advance Database Topics

In this lecture we will explore the advanced database techniques that power large scale systems. Replication for resilience, sharding for horizontal scaling and polyglot persistence for choosing the right database for each workload.

Scaling Strategies - SQL vs. NoSQL

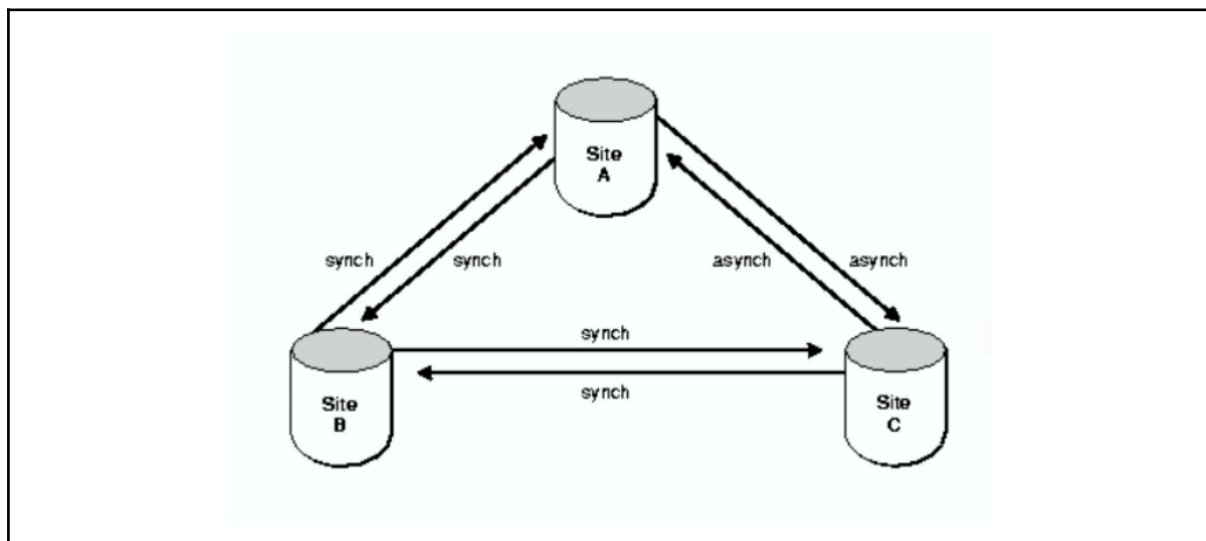
One of the most architectural differences between SQL and NoSQL databases is how they scale as demand grows. Traditional SQL databases typically scale vertically. When the database becomes a bottleneck, the common response is to make the server bigger and better, more CPU, more memory, faster storage. This works well because the architecture remains relatively simple, and you continue to benefit from strong asset guarantees and predictable consistency. The challenge however, is that every machine has a ceiling. Eventually, you hit hardware limits, upgrades become increasingly expensive, and the entire database can become a critical single point of failure.



NoSQL databases, on the other hand, were designed to solve the scalability scaling problem. Instead of building larger machines, they distribute data and traffic across many machines. This horizontal scaling model allows systems to grow almost incrementally when demand increases, making it a natural fit for internet-scale applications, global workloads, and massive datasets. It also improves resilience because now the system can continue operating even when individual nodes fail. The trade-off is complexity. Once data is distributed across multiple nodes, challenges like partitioning, replication, synchronization, and consistency become part of the architecture. This is why many NoSQL systems relax strict consistency guarantees and adopt eventual consistency to achieve higher availability and scalability.

As architects, the key lesson is that this isn't simply a SQL vs. NoSQL decision. It is a trade-off between simplicity and distributed scale. If consistency and transactional integrity dominate your requirements, vertical scaling with SQL is often sufficient. If your primary challenge is handling massive growth in users, traffic, or data volume, horizontal scaling becomes the more practical approach.

What is Replication?



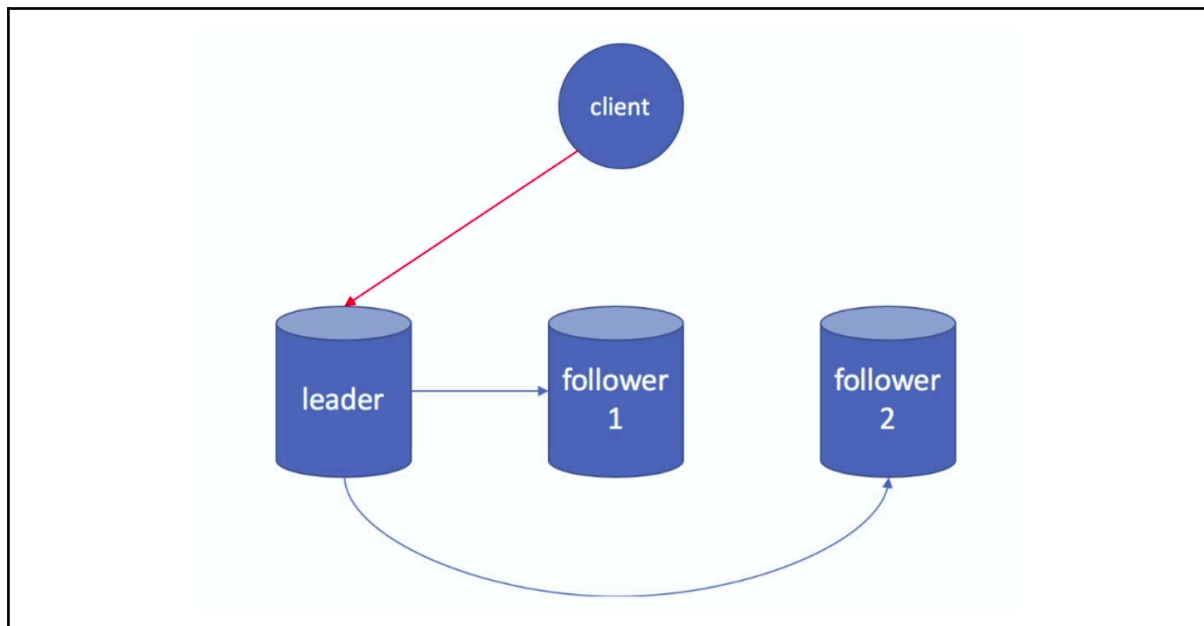
Replication exists because databases eventually become too important to rely on a single machine. If that machine fails, or becomes overloaded, or loses connectivity, the entire application is at risk. Replication solves this by maintaining copies of the same data across multiple database nodes. The immediate benefit is resilience. If one node becomes unavailable, another replica can continue serving the request, and thereby significantly improving fault tolerance and overall system availability.

Replication also helps with scale. In many applications read traffic far exceeds write traffic. By distributing read requests across multiple replicas, we can increase throughput without continuously upgrading a single database server. This is one of the most common techniques used by large-scale systems to handle growing demand. However replication introduces one of the fundamental trade-offs in distributed systems - consistency versus availability. The moment data exists on multiple nodes, keeping every copy perfectly synchronized becomes challenging, especially during network failures or partitions.

As a result, many real-world systems prioritize availability and allow replicas to be temporarily out of sync, leading to eventual consistency. As architects, the key question is now whether to use replication, but what consistency guarantees the business requires. Systems like banking may prioritize consistency, while social media, e-commerce and content platforms often accept slight replication delay in exchange for higher availability and better scalability.

Leader-Follower Replication

In this model, the leader acts as a single source of truth. Every write, whether is an insert, update, or deleted, is first committed to the leader. Those changes are then propagated to one or more followers, which primarily serve read requests. This allows the system to scale read traffic efficiently without overwhelming the primary database.

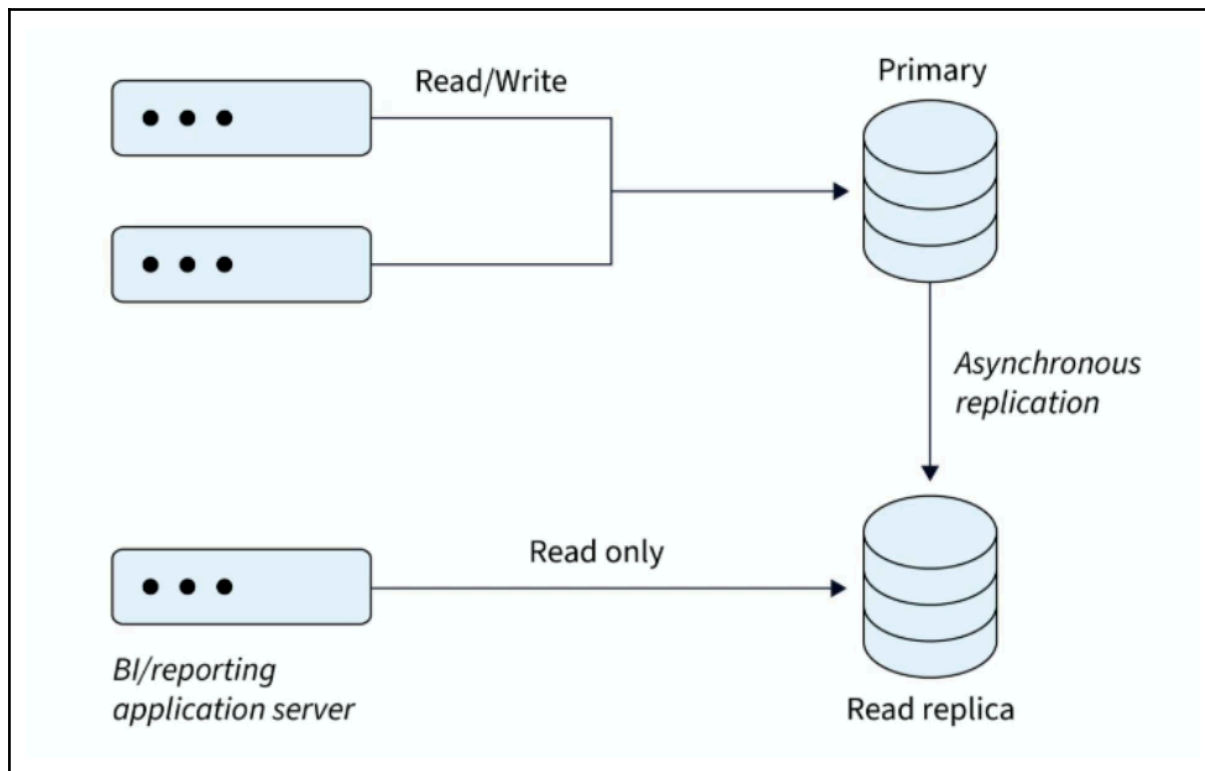


The architectural advantage is very clear. As read demand grows, we can add more followers and distribute queries across them. Many large-scale applications use this pattern because read workloads are often far more than write workloads. However, the design introduces an important consistency challenge. In most production systems, replication is asynchronous to avoid slowing the writes. That means there is a small delay between the changes being committed on the leader and the same change on followers. During this window, users reading from a follower may see some stale data. This is a classic distributed system trade-off. We gain scalability, performance, and availability, but we may sacrifice immediate consistency.

As architects we must decide whether the application can tolerate being slightly outdated reads or whether certain operations should always be read directly from the leader to guarantee the latest data. The right choice depends on the business requirement and the user expectations.

Read Replicas

Now read replicas are one of the simplest and most efficient ways to scale a database when the primary bottleneck is read traffic rather than writes. In many real-world applications, news websites, e-commerce, social media feeds, or streaming platforms, the number of reads can be hundreds or even thousands of times greater than the number of writes. If every request hits the primary database, it quickly becomes overloaded.



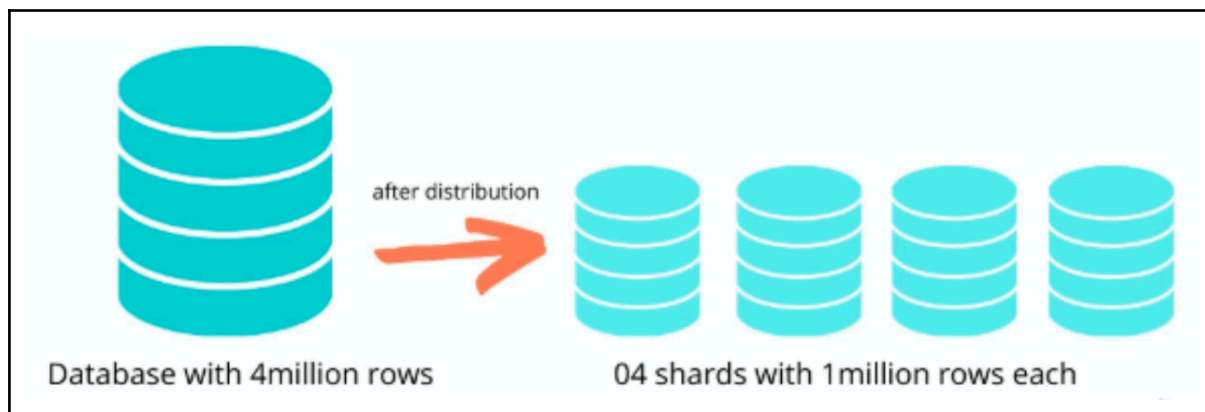
Read replicas solve this by creating additional copies of the data that can serve read requests independently. While read replicas are built using replication technology, their purpose is more focused than a traditional leader-follower setup. They exist primarily to distribute read traffic and improve the performance. All writes still go to the primary database, and the replicas simply stay synchronized with it and serve the read queries.

The major advantage is operational flexibility. During traffic spikes additional replicas can often be added with the minimal application changes, allowing the system to scale horizontally for reads without redesigning the database architecture. The trade-off remains consistency as replicas often update asynchronously causing delay before recent changes appear on them. That is why critical operations, such as viewing a just-updated account balance or order status, may still need to be read from the primary database.

As architects think of read replicas as a targeted scaling tool, they don't solve write bottlenecks, but they are often the fastest and most cost-effective way to increase database capacity when read traffic is the dominant workload.

What is Sharding?

Sharding is what organizations eventually turn to when a database can no longer be scaled effectively by replication or bigger hardware. At some point, the challenge is not just serving more requests, it's that the sheer volume of data and write traffic exceed what a single database node can handle.



The core idea is very simple. Instead of storing all data in one database, we partition it across multiple independent databases called shards. Each shard owns a portion of the dataset and processes only the requests related to that data. By distributing both storage and workload, the system can continue growth almost indefinitely by adding more shards. There are 3 types of sharding:

- **Horizontal Sharding:** This is the most common type of sharding. In this sharding, the rows are distributed across shards based on a sharding key. The key could be user ID, customer ID or region. This allows both storage and write throughput to scale as your dataset grows, making it a popular choice for large Internet-Scale applications.
- **Vertical Sharding:** Vertical sharding takes a different approach. Instead of spitting rows, it separates data by business function or access pattern. For example, user profiles, billing data, and analytics data may live in different databases. This is often useful when different parts of the application have very different performance requirements or growth patterns.

However, sharding is not a free scalability upgrade. Once the data is distributed, the system must determine where the data live, route requests to the correct shard, and handle operations that span across multiple shards.

As architects, sharding is typically considered a later-stage scaling strategy. Sharding helps you scale reads, but sharding is often the technique that allows you to scale storage capacity and write throughput beyond the limits of a single database node.

Sharding Strategies

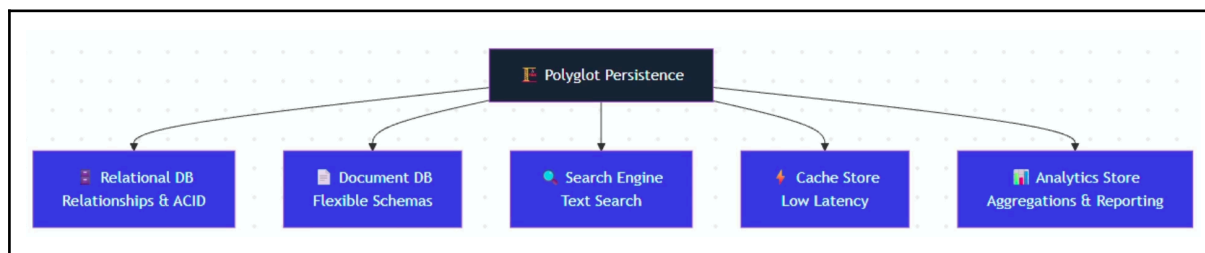
As systems scale, choosing the right sharding strategy becomes just as important as deciding to shard in the first place. The goal is always the same. Workload data and workload efficiently, but different strategies optimize for different requirements.

- **Range-based sharding** is often the easiest to understand and implement. Data is partitioned by a range of values, making range queries efficient because related records stay together. The challenge is uneven growth. If new data continuously falls into the same range, one shard can become overloaded while others remain lightly used, thus creating hotspots.
- **Hash-based sharding** addresses that imbalance by using a hash function to spread data more uniformly across shards. This typically produces better load distribution and prevents any single shard from becoming a bottleneck. The trade-off is that data loses its natural ordering, making range-based queries more complex and often requiring requests to multiple shards. A major limitation of traditional hashing appears when the cluster grows. Adding or removing a node changes the hash distribution, forcing large amounts of data to be redistributed.
- **Consistent hashing** was designed to solve this problem. By minimizing the data movement during topology changes, it allows systems to scale more gracefully and recover from node failures with far less disruption. This is one of the key techniques behind highly elastic distributed databases.
- **Geo-based sharding** takes a business-oriented approach by partitioning data according to users' location or region. Instead of optimizing purely for distribution, it optimizes for latency, regulatory compliance, and user experience. Keeping users close to their data often improves the performance and it simplifies data residency requirements too.

The important architectural lesson is that there is no universally best sharding strategy. Hash-based sharding optimizes query locality Hash-based sharding optimizes distribution Consistent hashing optimizes elasticity Geo-based sharding optimizes locality Mature distributed systems often combine multiple approaches to balance performance, scalability, and their operational simplicity.

Polyglot Persistence

Polyglot persistence is an architectural mindset that recognizes a very simple reality. Different data problems often require different database solutions. In smaller applications, it is common to store everything in a single database. But as systems grow, workloads become more diverse. The database that excels at transactional processing may not be the best choice for full-text search, analytics, caching, or high-volume event storage. Trying to force one database to handle every use case often leads to performance bottlenecks and architectural compromises.



Polyglot persistence addresses this by allowing each service or component to use a database that best matches its requirements. A relational database might manage transactional business data. A search engine can power product discovery. A document database can store flexible schemas and an analytics platform can process large-scale reporting workload.

The key advantage is optimization. Instead of accepting one-size-fits-all solution, each workload benefits from storage structure, indexing strategies, and query capabilities designed specifically for its needs. This often leads to better performance, improved scalability, and much more efficient resource utilization.

The trade-off is increased operational complexity. Multiple databases means multiple technologies to manage, monitor, secure, backup, and maintain. That is why architects should not adopt polyglot persistence simply because it is popular. The goal is not to use many databases but to solve different data challenges with the most appropriate technology. The rule of thumb is, start simple with one database when possible, and introduce additional data stores only when a clear scalability, performance, or business requirement justifies the added complexity. That is how polyglot persistence became a strategic architectural decision rather than a technology choice.

Summary & Key Takeaways

- Database Scaling strategies
- Replication and Replication strategies
- Sharding and Sharding Strategies
- Polyglot persistence boosts performance in complex systems
- What's next:
 - Object Storage in Modern Systems

Interview Questions - Advance Database Topics

- What is the difference between horizontal and vertical scaling? When would you prefer one over the other?
- Explain leader-follower replication. How does it impact consistency and availability?
- What are the pros and cons of using read replicas?
- Compare range-based and hash-based sharding. What are the trade-offs of each?
- Why is consistent hashing important in distributed databases?
- How does the CAP theorem influence the design of distributed databases? Can you give an example of a CP or AP system?
- What is polyglot persistence? Why might an architecture choose to use multiple types of databases?
- How do systems like Netflix or Uber use a mix of database technologies in production?